Neural Networks 187 (2025) 107405

Contents lists available at ScienceDirect

Neural Networks

journal homepage: www.elsevier.com/locate/neunet

Full Length Article

Mathematical expression exploration with graph representation and generative graph neural network☆

Jingyi Liu [a,b], Weijun Li [a,b,*], Lina Yu [a,b,*], Min Wu [a,b], Wenqiang Li [a], Yanjie Li [a,b], Meilan Hao [a,b]

[a] AnnLab, Institute of Semiconductors, Chinese Academy of Sciences, Beijing, 100083, China
[b] School of Integrated Circuits & Center of Materials Science and Optoelectronics Engineering, University of Chinese Academy of Sciences, 100049, Beijing, China

ARTICLE INFO

Keywords:
Symbolic regression
Directed acyclic graph
Graph neural network
Reinforcement learning

ABSTRACT

Symbolic Regression (SR) methods in tree representations have exhibited commendable outcomes across Genetic Programming (GP) and deep learning search paradigms. Nonetheless, the tree representation of mathematical expressions occasionally embodies redundant substructures. Representing expressions as computation graphs is more succinct and intuitive through graph representation. Despite its adoption in evolutionary strategies within SR, deep learning paradigms remain under-explored. Acknowledging the profound advancements of deep learning in tree-centric SR approaches, we advocate for addressing SR tasks using the Directed Acyclic Graph (DAG) representation of mathematical expressions, complemented by a generative graph neural network. We name the proposed method as ***Graph**-based **D**eep **S**ymbolic **R**egression (GraphDSR)*. We vectorize node types and employ an adjacent matrix to delineate connections. The graph neural networks craft the DAG incrementally, sampling node types and graph connections conditioned on previous DAG at every step. During each sample step, the valid check is implemented to avoid meaningless sampling, and four domain-agnostic constraints are adopted to further streamline the search. This process culminates once a coherent expression emerges. Constants undergo optimization by SGD and BFGS algorithms, and rewards refine the graph neural network through reinforcement learning. A comprehensive evaluation across 110 benchmarks underscores the potency of our approach.

1. Introduction

Symbolic Regression (SR) delineates a regression modality seeking the optimal mathematical expression congruent with a specified dataset. Specifically, given observed data points $\{\mathcal{X}, y\} \in \mathbb{R}^{n\times(d+1)}$, where n denotes the data count and d represents the dimensionality of input variables, SR discerns the function $y = f(\mathcal{X})$, amalgamating primitive operators like $\sin, \cos, +, -$, etc. SR has wide applications in material discovery (Wang, Wagner, & Rondinelli, 2019), physical formula discovery (Tenachi, Ibata, & Diakogiannis, 2023; Udrescu & Tegmark, 2021), etc.

SR deviates from conventional regression techniques in its dual-optimization objectives. While traditional methods, like polynomial fitting, SVM, and neural networks, presuppose a mathematical structure and focus on discerning optimized parameters (be it local or global), SR confronts a two-fold challenge: ascertaining the model's structural form while also determining its parameters. This duality in SR's objective renders it a particularly complex endeavor.

Genetic Programming (GP) using a binary tree representation has been a cornerstone of SR. By representing mathematical expressions through pre-order traversal of binary trees, methods like FFX (McConaghy, 2011) and SBP-GP (Virgolin, Alderliesten, & Bosman, 2019a) have attempted to evolve optimal mathematical models through genetic operations such as mutation and crossover. However, with the rise of deep learning, new methodologies have been explored to potentially enhance the efficiency and accuracy of SR: 1. EQL (Kim et al., 2021) replaces traditional neural network activation functions with basic mathematical operations such as $\sin, \cos, +$. This transformation attempts to make the network generate more symbolic-like expressions directly through its architecture; 2. Policy nets and reinforcement learning methods (Mundhenk et al., 2021; Petersen et al., 2021) apply a policy network predicts the categorical probability of each token, reinforcement learning is utilized to optimize the policy network's parameters. By rewarding the network for producing more accurate expressions, the network learns over time to produce better and more symbolic-like formulas; 3. Another avenue being explored

☆ Supported by the National Natural Science Foundation of China, No. 92370117 and CAS Project for Young Scientists in Basic Research (No. YSBR-090).
* Corresponding author at: AnnLab, Institute of Semiconductors, Chinese Academy of Sciences, Beijing, 100083, China.
E-mail addresses: liujingyi@semi.ac.cn (J. Liu), wjli@semi.ac.cn (W. Li), yulina@semi.ac.cn (L. Yu), wumin@semi.ac.cn (M. Wu), liwenqiang@semi.ac.cn (W. Li), liyanjie@semi.ac.cn (Y. Li), mlhao@semi.ac.cn (M. Hao).

https://doi.org/10.1016/j.neunet.2025.107405

Available online 21 March 2025
0893-6080/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

is training large models to directly learn the mapping between pre-order traversals and observed data (Li et al., 2023; Luca, Tommaso, Alexander, Aurelien, & Giambattista, 2021; Valipour, You, Panju, & Ghodsi, 2021). Essentially, these models aim to learn and represent complex mathematical expressions directly from data.

Beyond the studies of tree representation for SR tasks, graph-centric approaches have surfaced as an alternative paradigm. A notable subset of this is Cartesian Genetic Programming (CGP) (Miller, 2020). CGP represents computational frameworks as directed graphs, delineating their utilization through netlists. This methodology employs a netlist-derived address-centric genotype, comprised of integer pointers directed towards an array of rudimentary functions or data locales. The advantage of graph encoding over tree structures is the potential for reusable nodes and the capability for multiple outputs. While evolutionary strategy-based graph methods have demonstrated superiority over tree-based counterparts, limited research delves into the interplay of deep learning with graph-based SR techniques. Indeed, our prior research (Liu et al., 2023) juxtaposed SymNet (a specific graph representation of mathematical expressions) against the pre-order traversal of binary trees. Employing a consistent learning approach, graph-based representations outshined their counterparts in learning-from-scratch methods.

While SymNet delivers superior results, it is constrained by a static search space and becomes increasingly unwieldy and intractable as its architectural complexity scales. To address these limitations in symbolic regression, this study introduces a graph-based framework underpinned by a graph-generating neural network named ***Graph**-based **D**eep **S**ymbolic **R**egression (GraphDSR)*. We employ Directed Acyclic Graphs (DAGs) to encapsulate mathematical expressions, where each vertex is governed by a categorical probability distribution, and adjacency matrices delineate inter-node connections. A specialized graph neural network is deployed to produce both the vertex-specific categorical distributions and the adjacency matrix distributions. Utilizing these probability distributions as guidelines, we sequentially sample connections and vertex types until a comprehensive expression is assembled. Once the computational graph is established, constants are optimized using Stochastic Gradient Descent (SGD) and the Broyden–Fletcher–Goldfarb–Shanno (BFGS) (Fletcher, 2013) algorithm. Individual DAG connections are assigned constant values, and a risk-seeking policy gradient technique is employed for subsequent updates to the generative graph neural network. Crucially, the graph neural network's message-passing mechanism allows for the contextual influences of adjacent nodes to be seamlessly integrated into each vertex's determination.

Given the expansive search space inherent to the SR endeavor, consider the presence of m primitive functions, d variables, with an expression length capped at L. The resultant search space is thus computed as $(m + d)^L$. In navigating this vast expanse, it becomes prudent to discard redundant or nonsensical expressions. To this end, the adjacency matrix serves as a tool to verify expression legitimacy and regulate token probability during sampling. To encapsulate, our seminal contributions are:

- Utilization of a graph neural network to elucidate both the categorical probability distribution of node types and the Bernoulli distributions pertaining to the edges.
- The conceptualization of an expression generation methodology leveraging DAGs, complemented by constant optimization.
- The introduction of an efficient syntax verification technique ensures the graph embodies a syntactically valid mathematical expression whilst also refining the search space.

The paper's subsequent sections are structured thusly: Section 2 delves into the extant literature on SR. Section 3 elucidates the GraphDSR framework. Section 4 delineates our experimental approach and its outcomes. The culmination of our discourse and future work are presented in Section 5 and Section 6, respectively.

2. Related works

2.1. Tree representations

2.1.1. GP-based symbolic regression

The foundation of tree-based SR in GP can be traced back to John R. Koza's pioneering work (Koza, 1994). Koza's approach uses a node to represent an operation and each leaf to represent a terminal or variable. By evolving populations of these trees through genetic operators like mutation and crossover, the method could produce expressions that accurately fit given datasets. Recognizing the limitations of the traditional mutation where many mutations might be "neutral" (i.e., not affecting the program's output), SDM (Beadle & Johnson, 2009) focuses on the semantics or outputs of the programs, their mutation operation prioritized changes that genuinely affected the function of the tree, potentially speeding up the evolutionary process by avoiding unnecessary genetic changes. Considering the complex task can be split into several simpler tasks, MRGP (Arnaldo, Krawiec, & O'Reilly, 2014) evolving multiple simpler tree models that captured different facets of the data. These simpler models were then aggregated into a cohesive regression model, allowing for a more modular and potentially more interpretable approach to SR. Similar work includes PS-Tree (Zhang, Zhou, Qian, & Zhang, 2022). While not strictly a tree-based method, FFX (McConaghy, 2011) offered insights into a more deterministic and streamlined approach to SR. By evolving simple basis functions in sequence, it highlighted the potential advantages of fixed-sequence evolution over the more stochastic nature of traditional GP. More recently, AB-GEP (Lu, Xu, Luo, & Wang, 2022) incorporates multi-armed bandit problems in an adversarial setting, dynamically adjusting the selection pressure, balancing exploration and exploitation during the evolution. This makes the search process more efficient, helping to avoid premature convergence on suboptimal solutions.

A significant advancement in the realm of tree-based GP for SR was Pawlak and Krawiec (2018) They proposed a shift from the syntactic space (structures of programs) to the semantic space (outputs of programs). This move towards semantic operations, while building on the tree-based foundations, provided a more directed search mechanism, potentially smoothing the evolutionary process and enhancing convergence. EPLEX (La Cava, Spector, & Danai, 2016), Pawlak, Wieloch, and Krawiec (2014), SBP-GP (Virgolin, Alderliesten, & Bosman, 2019b), etc are such methods.

2.1.2. Deep learning-based symbolic regression

Recent developments in symbolic regression (SR) have seen a surge in the use of deep learning techniques, often outperforming traditional GP-based methods. Broadly, these deep learning approaches can be categorized as learning-from-scratch and learning-with-experience (Liu et al., 2023).

Learning-from-scratch methods typically involve training a network from the ground up, yielding the best expression post-training. A notable example is DSR (Petersen et al., 2021), which employs an RNN to produce probability distributions for each token, with risk-seeking policy gradients steering the RNN towards more optimal expressions. An extension of DSR, DSO (Mundhenk et al., 2021), merges GP techniques with DSR, where the RNN serves as an initializer for GP algorithms. There is also DGSR-MCTS (Kamienny, Lample, Lamprier, & Virgolin, 2023), which opts for Monte-Carlo-Tree-Search instead of the traditional GP algorithm.

On the other hand, large-scale pre-trained models are superior in terms of accuracy, especially for expressions encountered during training or those bearing structural similarities. Their inference speeds also tend to be fast. An instance is SymbolicGPT (Valipour et al., 2021), which translates expressions into text sequences, leveraging T-Net for data encoding and GPT for decoding. Here, the encoder transforms data points into features, which the decoder then uses to formulate

expressions. Moreover, NeSymReS (Luca et al., 2021) adopts the pre-order traversal of binary trees, making training easier by bypassing the need to discern the syntactic essence of each token. Li et al. (2023) suggest utilizing a contrastive loss to enhance data feature distinction. Additionally, SymFormer (Vastl, Kulhánek, Kubalík, Derner, & Babuska, 2022) introduces a constant encoding approach, providing a superior starting point for constant optimization.

2.2. Graph representations

Cartesian Genetic Programming (CGP) is a notable example of using graph representations in symbolic regression and other optimization problems. In CGP, the genotype is a fixed-length linear genome that is mapped into a phenotype which is typically a Directed Acyclic Graph (DAG). This representation allows for more flexibility and redundancy, which can be beneficial in terms of evolutionary search. The nodes in the graph represent functions (e.g., mathematical operations), and the edges represent the flow of data. Based on the standard CGP algorithm, a lot of variants were developed concerning the different aspects. For example, Modular CGP (Walker & Miller, 2004, 2008) introduces the concept of modularity into the representation, the evolved programs can have modules (or sub-programs) that can be called multiple times. This can be seen as a way to encapsulate and reuse functionality, similar to how functions or procedures are used in traditional programming. To get a detail of the improved algorithms, please refer to the review (Miller, 2020).

Another notable branch is the use of graph networks to learn symbolic physics (Cranmer, Xu, Battaglia, & Ho, 2019). This method involves training a GNN to predict a symbolic expression given a set of input–output pairs. The graph representation aids in capturing the hierarchical and compositional nature of mathematical expressions. Similar work includes EQL (Kim et al., 2021) which exchanges the activation functions of neural networks with primitive functions and then uses the sparse technique to learn the constant by SGD. These two methods set a fixed meta-structure, and then prune the graph to drop redundant structures by constants approaching zero.

3. Method

3.1. Framework

Leveraging a graph neural network as the probabilistic graph generator, we generate the probabilistic distributions for nodes and edges within mathematical expressions, as depicted in Fig. 1. These mathematical graphs evolve incrementally, with each new node and edge introduced based on previously determined DAG. Upon completion of the sampling, we employ a dedicated optimizer to ascertain the constant values. To compensate for the gradients obscured during the sampling phase, reinforcement learning services refine the graph neural networks. Notably, we opt for risk-seeking policy gradients, given their proven efficacy for SR tasks (Petersen et al., 2021).

3.2. Representation of graphs and the generation process

Mathematical expressions are portrayed through the Directed Acyclic Graphs (DAGs). Our objective is to discern a distribution $\mathbb{P}(\mathcal{G})$ over DAGs representing valid mathematical expressions. By "valid", we infer that the expressions derived from these sampled DAGs adhere to fundamental expression syntax and other stipulated constraints. A DAG is characterized as a tuple $\mathcal{G} = (A, X)$. Here, $A \in \mathbb{R}^{N\times N}$ denotes the $N \times N$ adjacency matrix where $A_{ij} = 1$ signifies node i being an input to node j. Conversely, X represents an attribute vector of size N that encodes the operation type. For any given operation i, its operational type X_i is restricted to a predetermined operator library of size D, for instance, $\{+, -, \times, \div, \sin\}$.

In the realm of graph structures, maintaining flexibility invariably presents a challenge in constraining the number of outputs; the introduction of any ensuing nodes potentially assumes the role of the output. Thus, our research pursuits commence with a singular output in consideration. To guarantee that the resultant DAGs manifest solely one output, we meticulously impose a stipulation on A to exhibit a lower triangular form during the intricate phase of sampling (as delineated in Fig. 2). This sampling paradigm encompasses myriad steps; within each juncture, we select a row from the adjacency matrix. Upon reaching the maximum node count, node selection culminates, and indispensable edges are interwoven to craft a holistic expression. In the wake of this enhancement, the upper triangular section of A is subsequently secured.

In essence, our methodology involves 1. generating a node X_1; 2. examining potential edges $A_{1,:} \in \mathbb{R}^2$ to assemble a tentative edge set; 3. selecting edges from this set while incorporating a blank new node; 4. ascertaining the node type X_2 for this new node. This cycle from steps 2–4 is reiterated until either all nodes meet their arity requirements or the node count hits its threshold. Our strategy crafts every entry in a single row in one iteration, contingent upon the previously formed graph. Consider the illustrative embodiment depicted in Fig. 2, where the advanced GNNs initially opt for a node denoted by +, in conjunction with the edge $(2, 1)$, these are discerningly picked from the second row of the adjacency matrix $A_{2,:}$. At this juncture, the node X_2 has not yet been identified. Thereafter, in the ensuing stride, X_2 is discerningly chosen as $\sin$, thence the connections $(3, 1), (3, 2)$ are definitively established. This methodology perseveres in its recurrent execution until a legitimate mathematical expression $\sin x^4 + x^4$ is astutely selected.

3.3. Probabilistic generators for DAGs

3.3.1. Auto-regression generation

The approach utilized is the contemporary deep auto-regressive graph generative model as explained in the literature (Liao et al., 2019). The DAGs $\mathcal{G}$ are sampled row-wise and node-wise from the adjacency matrix A and node type X. Therefore, the joint probability of X and A is ascertained by:

$$\mathbb{P}(A, X) = \prod_{i=1}^{N} \mathbb{P}(A_{i,:}|A_{i-1,:}, X_{i-1}, \dots, A_{1,:}, X_1) \mathbb{P}(X_i|A_{i-1,:}, X_{i-1}, \dots, A_{1,:}, X_1) \tag{1}$$

where $A_{i,:}$ and X_i denote the ith row of the adjacency matrix A and the ith operation type, respectively. The condition distributions $\mathbb{P}(A_{i,:}|A_{i-1,:}, X_{i-1}, \dots, A_{1,:}, X_1)$ and $\mathbb{P}(X_i|A_{i-1,:}, X_{i-1}, \dots, A_{1,:}, X_1)$ define the probability of generating the current edges and node type conditioned on the existing graph, respectively. We use GNNs rather than RNNs to avoid long-term bottlenecks as well as improve conditioning (Liao et al., 2019). The conditional distributions are calculated as follows:

$$\mathbb{P}(A_{i,:}|A_{i-1,:}, X_{i-1}, \dots, A_{1,:}, X_1) = \sum_{i=1}^{K} \alpha_k \prod_{1\le j<i} \theta_{k,i,j} \tag{2}$$

$$\mathbb{P}(X_i|A_{i-1,:}, X_{i-1}, \dots, A_{1,:}, X_1) = \beta_1, \dots, \beta_D \tag{3}$$

$$\alpha_1, \dots, \alpha_K = \mathrm{Softmax}(\sum_{1\le j<i} \mathrm{MLP}_\alpha(h_i^S - h_j^S)) \tag{4}$$

$$\beta_1, \dots, \beta_D = \mathrm{Softmax}(\mathrm{MLP}_\beta(h_i^S)) \tag{5}$$

$$\theta_{1,i,j}, \dots, \theta_{K,i,j} = \mathrm{Sigmoid}(\mathrm{MLP}_\theta(h_i^S - h_j^S)) \tag{6}$$

where the distributions of the operation type and edges are categorical and K-mixture of Bernoulli respectively. D is the number of basic operators. MLP_α, MLP_β, MLP_θ are different instances of two-layer MLPs with ReLU activations. Here, h_i^S is the representation of ith node returned by a GNN which has executed S steps of message passing at each generation step.

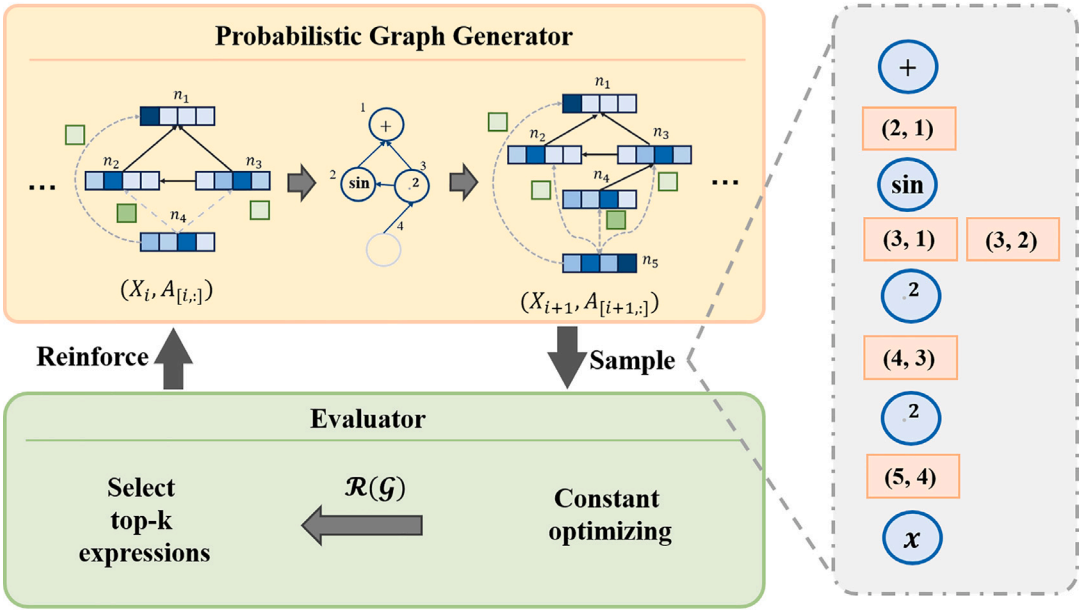

Fig. 1. Framework of GraphDSR. The probabilistic graph generator produces node and edge distributions step by step through a graph neural network. The sampled expressions are then fed to a constant optimizer to obtain the constants. Expressions are sorted by the rewards and the top-k expressions are selected to update the graph neural network. The blue circle in the gray box represents the operator and the orange box represents the edge from node i to j.

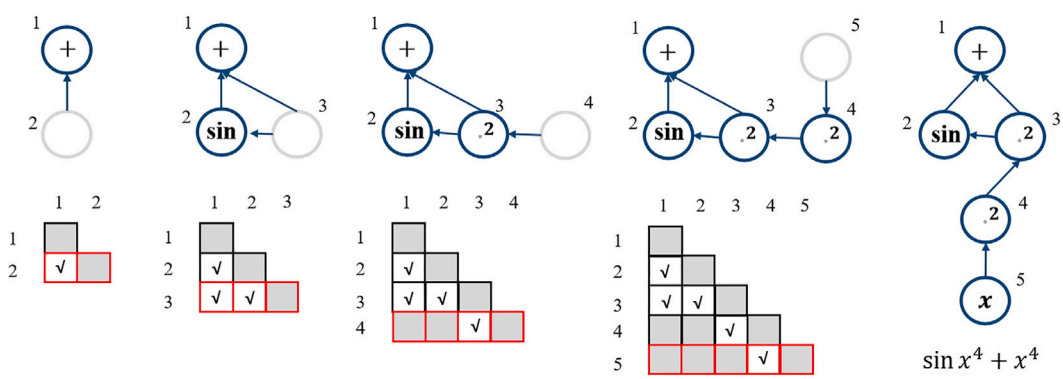

Fig. 2. Procedure for sampling a DAG representative of the expression $\sin x^4 + x^4$. Gray-highlighted boxes delineate unsampleable connections between node i and j. Conversely, the crimson-bordered boxes delineate the probabilistic distribution of edges between node i and its antecedents. The methodology commences with the extraction of a foundational node, followed by the evaluation of potential connective edges, the subsequent sampling of these edges, and finally, the categorization of the node type. This sequence persists until the node input tally aligns with their specified arities.

3.3.2. Message passing GNNs

For each defining step $n \leq N$, within the autoregression generation process delineated above, we rely upon the representation of nodes up to and inclusive of n itself. We employ the utilization of message propagation GNNs (Gilmer, Schoenholz, Riley, Vinyals, & Dahl, 2017), incorporating an attention mechanism analogous to Ref. Liao et al. (2019), to derive these node representations $\{h_i^S\}$. In particular, the execution of the sth message propagation stage involves the successive application of the appended equations,

$$m_{ij}^s = f(h_i^s - h_j^s) \tag{7}$$

$$\hat{h}_i^s = [h_i^s, u_i] \tag{8}$$

$$a_{ij}^s = \mathrm{Sigmoid}(g(\hat{h}_i^s - \hat{h}_j^s)) \tag{9}$$

$$h_i^{s+1} = \mathrm{GRU}(h_i^s, \sum_{j \in \mathcal{N}(i)} a_{ij}^s m_{ij}^s) \tag{10}$$

where, f, g are two-layer MLPs. $\mathcal{N}(i)$ symbolizes the set of node i and its neighboring nodes. m_{ij}^s specifies the message conveyed from node i to j at the sth message propagation stage. The connectivity that dictates propagation in GNNs is described by $A_{1:i-1,1:i-1}$. Given that the last node $A_{i,:}$ has yet to be generated, it is fully connected. The multi-turn message propagation S during each defining phase is executed to capture the structural dependency among nodes and edges. u_i delineates an auxiliary feature vector that assists in differentiating the ith node from others. We employ the one-hot encoding method and initialize the hidden node representation h_i^0 as the corresponding one-hot model. This is passed through a projective layer as represented by the following equation,

$$h_i^0 = W\,\mathrm{OneHot}(X_i) + b \tag{11}$$

3.3.3. Sampling constraints

SR poses a challenge as an NP-hard problem. The expansive search space escalates dramatically as we augment the number of primitive operators, variables, and the permissible expression length. Consequently, streamlining this search space emerges as an indispensable technique to not only enhance fitting accuracy but also optimize time complexity. Our method incorporates domain-agnostic constraints to realize this objective: 1. The initial node is mandated to be an operator;

2. The graph's node number must not surpass a predefined maximum; 3. The appearance of inverse functions, exemplified by $\log(\exp(\cdot))$, is prohibited; 4. Nesting of trigonometric functions is restrained due to the absence of such composite trigonometric operations in a majority of scientific realms (Petersen et al., 2021). Throughout the DAG creation, these constraints are meticulously applied in every step. This is accomplished by nullifying the probabilities associated with non-preferred node categories and their corresponding edges.

To elucidate this concept further, let us inspect the DAG construction by invoking these constraints, showcased in Fig. 2. 1. Upon the generation of the first node, variables are rendered non-viable. As the graph remains acyclic, diagonal pathways of the upper triangular matrix are not viable edge candidates. Consequently, edge $(2,1)$ emerges as the selected option. 2. Node X_2 gets earmarked as $\sin$, and simultaneously, edges $(3,1),(3,2)$ are chosen. 3. The node X_3 is sampled as $\cdot^2$. Given that the arities of X_1 and X_2 are met, edges $(4,1)$ and $(4,2)$ get excluded (indicated by a gray box in the figure). Moreover, since the predecessors of node X_4 incorporate the trigonometric function $\sin$, ensuing nodes are barred from adopting trigonometric functions. This leads to the zeroing out of probabilities for types $\sin$ and $\cos$ 4. Node X_4 is persistently labeled as $\cdot^2$. Mirroring the previous step, only edge $(5,4)$ emerges as selectable. 5. Conclusively, node X_5 is identified as x, rendering the DAG as a full-fledged expression and concluding the process.

3.4. Reward function and constant optimization

Upon completion of node type X sampling and the adjacent matrix A initialization, we materialize the expression and assess it using a specified reward function. For a designated dataset $\{\mathcal{X}, y\}$, comprising n data points and the delineated expression f, the reward is determined as:

$$\tilde{R}(\mathcal{G}) = \frac{1}{(\mathrm{NRMSE}+1)} \tag{12}$$

$$\mathrm{NRMSE} = \frac{1}{\sigma_y}\sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - f(\mathcal{X}_i))^2} \tag{13}$$

where NRMSE is the root-mean-square error. We use depth-first traversal of a graph to get the mathematical expressions and optimize the constants $const$ in the expression by maximizing the reward function:

$$const^* = \arg\max_{const} \tilde{R}(\mathcal{G}; const) \tag{14}$$

3.5. Update with the risk-seeking policy gradient

Given the parameter of GNNs denoted as θ, we must note that the reward function is non-differentiable in terms of θ. This is primarily attributed to the fact that the expression sampling procedure is inherently non-amenable to backpropagation. Therefore, reinforcement learning has been harnessed to refine the GNN. The standard policy gradient is designed to augment the expected reward which deviates from the objective of SR. The primary focus of SR is to pinpoint the most fitting expression that aligns with the data, rather than merely elevating the average reward across all expressions. In light of this, the risk-seeking policy, as delineated by Petersen et al. (2021), is invoked:

$$J_{\mathrm{risk}}(\theta;\epsilon) \doteq \mathbb{E}_{\mathcal{G}\sim p(\mathcal{G}|\theta)}[\tilde{R}(\mathcal{G})|\tilde{R}(\mathcal{G}) \geq \tilde{R}_\epsilon(\theta)], \tag{15}$$

where $\tilde{R}_\epsilon(\theta)$ is characterized as the $(1-\epsilon)$-quantile of the reward distribution under the prevailing policy. This objective aims to amplify the rewards of the top ϵ fraction derived from the distribution, while concurrently disregarding samples beneath this delineated threshold. The gradient of $J_{\mathrm{risk}}(\theta;\epsilon)$ can be represented as:

$$\begin{aligned}\nabla_\theta J_{\mathrm{risk}}(\theta;\varepsilon) = \mathbb{E}_{\mathcal{G}\sim p(\mathcal{G}|\theta)}\Big[&(R(\mathcal{G}) - R_\varepsilon(\theta))\cdot \\ &\nabla_\theta \log p(\mathcal{G}|\theta) \mid R(\mathcal{G}) \geq R_\varepsilon(\theta)\Big]\end{aligned} \tag{16}$$

Employing a rudimentary Monte Carlo approximation of the gradient with a sample of size N:

$$\begin{aligned}\nabla_\theta J_{\mathrm{risk}}(\theta;\varepsilon) \approx \frac{1}{\varepsilon N}\sum_{i=1}^{N}\Big\{ &\Big[R(\mathcal{G}^{(i)}) - \tilde{R}_\varepsilon(\theta)\Big]\cdot \\ &\mathbf{1}_{R(\mathcal{G}^{(i)})\geq \tilde{R}_\varepsilon(\theta)}\nabla_\theta \log p(\mathcal{G}^{(i)}|\theta)\Big\},\end{aligned} \tag{17}$$

where $\tilde{R}_\epsilon(\theta)$ symbolizes the empirical $(1-\epsilon)$-quantile of the reward batch, and the function $\mathbf{1}_x$ yields a value of 1 when the condition x is satisfied, and 0 in its absence.

3.6. Computation efficiency analysis

In this section, we will analyze the time complexity of four types of SR methods: 1. Genetic Programming-based Symbolic Regression (GPSR); 2. Bayesian statistics framework BSR; 3. Reinforcement Learning-based end-to-end Symbolic Regression (RLSR) method DSR; and 4. large-scale pre-trained models.

Time complexity analysis of GPSRs, BSR, and RLSRs. These three types of algorithms share a similar sub-process when handling SR tasks. It can be split into three phases: expression generation, expression evaluation, and agent update. Note that, the agent update is different: genetic operation for GPSR, probability update for BSR, and policy net update for RLSRs. Assume we generate P expressions in each iteration, and the maximum length of the expression is L, then the time complexity of one expression evaluation is $o(\log L)$. The generated expressions will not always reach the maximum length, thus we take the log function. Let T be the maximum iteration times, then the overall time complexity of these methods could be $o(T \times P \times (G + \log L + A))$, where $o(G)$ is the time complexity of expression generation, and $o(A)$ is the time complexity of the agent update for each algorithm.

Normally, RLSRs utilize a small policy net, thus, we have $o(G_{\mathrm{GP}}) < o(G_{\mathrm{RLSR}}) < o(G_{\mathrm{BSR}})$ and $o(A_{\mathrm{GP}}) < o(A_{\mathrm{RLSR}}) < o(A_{\mathrm{BSR}})$. The Bayesian statistic framework suffers from complex statistical computation. As a reinforcement learning-based algorithm, the time complexity of our method is relatively high. Moreover, we use the graph neural network that contains inner message passing steps when sampling expressions, and 3 MLP are used, thus, the time complexity is higher than the origin reinforcement learning-based method DSR.

Time complexity of large-scale pre-trained models. For the pre-trained models, once the models are finished training, the time complexity of the search process is $o(1)$. The model only needs to conduct one propagation and beam search to obtain predicted expressions. They can be executed in a few seconds.

Discussion. Considering the mainstream applications of SR algorithms, such as material discovery (Wang et al., 2019), physical law discovery (Udrescu et al., 2020; Udrescu & Tegmark, 2020), etc., which are time-insensitive tasks: it is acceptable for a scientist to spend a few months to discover a natural rule. Therefore, we think the time complexity of our method is acceptable based on our better performances (see Sections 4.4.2 and 4.4.3). In addition, while the large-scale pre-trained models also exhibit similar performances as ours on public benchmarks (especially the SymFormer) and enjoy incomparably fast expression prediction time, they are limited by the training datasets and suffer from poor adaptability.

4. Experiments

4.1. Datasets

In our empirical investigations, we utilized an exhaustive collection of publicly available benchmarks that are universally acknowledged in the realm of SR. We subjected the GraphDSR to rigorous evaluation, employing 60 publicly available expressions sourced from a diverse array of SR benchmarks, including Nguyen (Uy, Hoai, O'Neill, McKay, & Galván-López, 2011), GrammarVAE, Jin, Neat, R (Krawiec & Pawlak,

Table 1
Details of public benchmarks.

Expression	Equation	Interval
Nguyen1	$x^3 + x^2 + x$	$x \in [-1, 1]$
Nguyen2	$x^4 + x^3 + x^2 + x$	$x \in [-1, 1]$
Nguyen3	$x^5 + x^4 + x^3 + x^2 + x$	$x \in [-1, 1]$
Nguyen4	$x^6 + x^5 + x^4 + x^3 + x^2 + x$	$x \in [-1, 1]$
Nguyen5	$\sin(x^2)\cos(x) - 1$	$x \in [-1, 1]$
Nguyen6	$\sin(x) + \sin(x + x^2)$	$x \in [-1, 1]$
Nguyen7	$\log(x + 1) + \log(x^2 + 1)$	$x \in [-1, 1]$
Nguyen8	$\sqrt{x}$	$x \in [0, 4]$
Nguyen9	$\sin(x) + \sin(y^2)$	$x, y \in [-1, 1]$
Nguyen10	$2\sin(x)\cos(y)$	$x, y \in [-1, 1]$
Nguyen11	x^y	$x, y \in [0, 1]$
Nguyen12	$x^4 - x^3 + 0.5y^2 - y$	$x, y \in [-1, 1]$
Nguyen2_	$4x^4 + 3x^3 + 2x^2 + x$	$x \in [-1, 1]$
Nguyen5_	$\sin(x^2)\cos(x) - 2$	$x \in [-1, 1]$
Nguyen8_	$\sqrt[3]{x}$	$x \in [0, 4]$
Nguyen8_2	$\sqrt[3]{x^2}$	$x \in [0, 4]$
Nguyen1_c	$3.39x^3 + 2.12x^2 + 1.78x$	$x \in [-1, 1]$
Nguyen5_c	$\sin(x^2)\cos(x) - 0.75$	$x \in [-1, 1]$
Nguyen7_c	$\log(x + 1.4) + \log(x^2 + 1.3)$	$x \in [-1, 1]$
Nguyen8_c	$\sqrt{1.23x}$	$x \in [0, 4]$
Nguyen10_c	$\sin(1.5x)\cos(0.5y)$	$x, y \in [-1, 1]$
GrammarVAE1	$\frac{1}{3} + x + \sin(x)$	$x \in [-1, 1]$
Jin1	$2.5x^4 - 1.3x^3 + 0.5y^2 - 1.7y$	$x, y \in [-3, 3]$
Jin2	$8.0x^2 + 8.0y^3 - 15.0$	$x, y \in [-3, 3]$
Jin3	$0.2x^2 + 0.5y^3 - 1.2y - 0.5x$	$x, y \in [-3, 3]$
Jin4	$1.5e^x + 5.0\cos(y)$	$x, y \in [-3, 3]$
Jin5	$6\sin(x)\cos(y)$	$x, y \in [-3, 3]$
Jin6	$1.35xy + 5.5\sin(x - 1.0)(y - 1.0)$	$x, y \in [-3, 3]$
Neat8	$\frac{e^{-(x-1)^2}}{1.2+(y-2.5)^2}$	$x, y \in [0.3, 4]$
Neat9	$\frac{1}{1+x^{-4}} + \frac{1}{1+y^{-4}}$	$x, y \in [-5, 5]$
R1	$\frac{(x+1)^3}{x^2-x+1}$	$x \in [-2, 2]$
R2	$\frac{x^5-3x^3+1}{x^2+1}$	$x \in [-2, 2]$
R3	$\frac{x^6+x^5}{x^4+x^3+x^2+x+1}$	$x \in [-1, 0.5]$
Livermore-4	$\ln(x + 1) + \ln(x^2 + 1) + \ln(x)$	$x \in [0.1, 2.2]$
Livermore-5	$x^4 - x^3 + x^2 - y$	$x \in [0, 1.2], y \in [-1, 1]$
Livermore-9	$x^9 + x^8 + x^7 + x^6 + x^5 + x^4 + x^3 + x^2 + x$	$x \in [-1.2, 0.6]$
Livermore-11	$\frac{x^2x^3}{x+y}$	$x \in [-1, 1], y \in [-10, 10]$
Livermore-12	$\frac{x^3}{y^3}$	$x, y \in [-2, 2]$
Livermore-14	$x^3 + x^2 + x + \sin(x) + \sin(x^2)$	$x \in [-4.4, 4.4]$
Livermore-15	$x^{\frac{1}{5}}$	$x \in [0, 1]$
Livermore-16	$x^{\frac{2}{5}}$	$x \in [0, 1]$
Livermore-17	$4\sin(x)\cos(y)$	$x, y \in [-5, 5]$
Livermore-18	$\sin(x^2)\cos(x) - 5$	$x \in [-2, 2]$
Livermore-19	$x^5 + x^4 + x^2 + x$	$x \in [-1, 1]$
Livermore-20	$\exp(-x^2)$	$x \in [-2, 2]$
Livermore-21	$x^8 + x^7 + x^6 + x^5 + x^4 + x^3 + x^2 + x$	$x \in [-1.2, 0.7]$
Livermore-22	$\exp(-0.5x^2)$	$x \in [-4, 4]$
Koza2	$x^5 - 2x^3 + x$	$x \in [-2, 2]$
Koza3	$x^6 - 2x^4 + x^2$	$x \in [-1.2, 1.2]$
Keijzer3	$0.3x\sin(2\pi x)$	$x \in [-1, 1]$
Keijzer4	$x^3\exp(-x)\cos(x)\sin(x)(\sin(x^2)\ \cos(x) - 1)$	$x \in [-2, 5]$
Keijzer6	$\frac{x(x+1)}{2}$	$x \in [-2, 1]$
Keijzer7	$\ln(x)$	$x \in [0.1, 4]$
Keijzer9	$\ln(x + \sqrt{x^2 + 1})$	$x \in [-6, 6]$
Keijzer11	$xy + \sin((x - 1)(y - 1))$	$x \in [-3, 3]$
Keijzer14	$\frac{8}{2+x^2+y^2}$	$x \in [-0.5, 0.5]$
Keijzer15	$\frac{x^3}{5} + \frac{y^2}{2} - y - x$	$x \in [-2, 2], y \in [0, 2]$

2013), Livermore (Petersen et al., 2021), Koza (Koza, 1994), Keijzer (Keijzer, 2003). In addition, we selectively incorporated equations comprising one to three variables from the AI Feynman (Udrescu & Tegmark, 2020) repository. The AI Feynman database contains many real-world physics equations. It is noteworthy that with the exception of the Nguyen benchmark, wherein intervals are configured as per DSR, the intervals in other benchmarks were calibrated to delineate an entirely smooth curve, the zenith of which is in proximate accordance with its nadir. Comprehensive details of the selected expressions can be found in Tables 1 and 2, which contain the expression name, corresponding equation, and the sampling interval.

4.2. Evaluating

We use training data to compute the reward for each candidate expression and use test data to evaluate the best-found candidate expression at the end of training. The number of training data and test data are the same. We use the Coefficient of Determination (R^2) to assess the quality of fitting results. The R^2 is expressed as follows:

$$R^2(y,\hat{y}) = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}, \tag{18}$$

where y_i and $\hat{y}_i$ are the ground-truth and predicted values for point i, respectively. $\bar{y}$ is the average of y_i over all points. n is the number of test points. The advantage of using R^2 is its simple interpretation. $R^2 > 0$ implies that the predicted value is better than the average value; $R^2 = 1$ implies that the algorithm obtained the perfect fit; and $R^2 < 0$ implies that the predicted value is worse than the average value. Each expression is conducted 10 independently run. Maximum R^2 and mean R^2 were calculated to evaluate the algorithm performances.

4.3. Experiment settings

4.3.1. Compared methods

Seven different methods were compared to investigate the proposed MSR's effectiveness.

- **Genetic programming (GP)**: a canonical implementation of GP with a standard crossover as well as subtree, hoist, and pointwise mutations.
- **BSR** (Jin, Fu, Kang, Guo, & Guo, 2019): a Bayesian statistics framework that exhibits advantages in better model interpretability, simpler ways to incorporate prior knowledge, and more cost-effective memory usage.
- **GP-GOMEA** (Virgolin, Alderliesten, Witteveen, & Bosman, 2021): a genetic programming algorithm that uses linkage learning to improve symbolic regression.
- **SBP-GP** (Virgolin et al., 2019a): a genetic programming algorithm that uses Linear Scaling (LS) in synergy with Semantic Backpropagation (SB) and within library search.
- **NeSymReS** (Luca et al., 2021): a Transformer-based large-scale pre-trained model that uses pre-order traversal as expression's encoding. It uses an encoder to extract features from variables and values, and a decoder to predict the skeleton of an expression. Finally, the constant values are optimized using BFGS.
- **SymFormer** (Vastl et al., 2022): similar to NeSymRes, but it uses a special encoding of constants to make accurate predictions. Constant optimization is based on the predicted constant values.
- **DSR** (Petersen et al., 2021): a learning-from-scratch method that uses an RNN emits the categorical probability distributions of each token, and risk-seeking policy gradients is applied to update the RNN.

Table 2
Details of Feynman benchmarks.

Feynman	Equation	Interval
I.6.20a	$f = e^{-\theta^2/2}/\sqrt{2\pi}$	$\theta \in [1,3]$
I.6.20	$e^{-\frac{\theta^2}{2\sigma^2}}/\sqrt{2\pi\sigma^2}$	$\theta, \sigma \in [1,3]$
I.6.20b	$f = e^{-\frac{(\theta-\theta_1)^2}{2\sigma^2}}/\sqrt{2\pi\sigma^2}$	$\theta, \theta_1, \sigma \in [1,3]$
I.10.7	$m = \frac{m_0}{\sqrt{1-\frac{v^2}{c^2}}}$	$m_0 \in [1,5], y \in [1,2], c \in [3,10]$
I.12.1	$F = \mu N_n$	$\mu, N_n \in [1,5]$
I.12.4	$E_f = \frac{q_1}{4\pi\epsilon r^2}$	$q_1, r, \epsilon \in [1,5]$
I.12.5	$F = q_2 E_f$	$q_2, E_f \in [1,5]$
I.14.3	$U = mgz$	$m, g, z \in [1,5]$
I.15.10	$p = \frac{m_0 v}{\sqrt{1-v^2/c^2}}$	$m_0 \in [1,5], v \in [1,2], c \in [3,10]$
I.16.6	$v_1 = \frac{u+v}{1+uv/c^2}$	$u, v, c \in [1,5]$
I.18.12	$\tau = rF\sin\theta$	$r, F \in [1,5], \theta \in [0,5]$
I.25.13	$V_e = \frac{q}{c}$	$q, c \in [1,3]$
I.27.6	$f_f = \frac{1}{\frac{1}{d_1}+\frac{n}{d_2}}$	$d_1, n, d_2 \in [1,5]$
I.29.4	$k = \frac{\omega}{c}$	$\omega, c \in [1,10]$
I.30.3	$I *= I *_0 \frac{\sin^2(n\theta/2)}{\sin^2(\theta/2)}$	$I *_0, n, \theta \in [1,5]$
I.34.10	$\omega = \frac{\omega_0}{1-v/c}$	$\omega_0 \in [3,10], v \in [1,2], c \in [1,5]$
I.34.14	$\omega = \frac{1+v/c}{\sqrt{1-v^2/c^2}}\omega_0$	$\omega_0 \in [3,10], v \in [1,2], c \in [1,5]$
I.34.27	$E = \hbar\omega$	$\hbar, \omega \in [1,5]$
I.37.4	$I_* = I_1 + I_2 + 2\sqrt{I_1 I_2}\cos\delta$	$I_1, I_2, \delta \in [1,5]$
I.39.10	$E = \frac{3}{2}p_F V$	$p_F, V \in [1,5]$
I.39.11	$E = \frac{1}{\gamma-1}p_F V$	$\gamma, V \in [2,5], p_F \in [1,5]$
I.43.31	$D = \mu_e k_b T$	$\mu_e, k_b, T \in [1,5]$
I.48.20	$E = \frac{mc^2}{\sqrt{1-v^2/c^2}}$	$m \in [1,5], c \in [1,2], v \in [3,10]$
II.3.24	$F_E = \frac{p}{4\pi r^2}$	$p, r \in [1,5]$
II.4.23	$V_e = \frac{q}{4\pi\epsilon r}$	$q, \epsilon, r \in [1,5]$
II.8.7	$E = \frac{3}{5}\frac{q^2}{4\pi\epsilon d}$	$q, \epsilon, d \in [1,5]$
II.8.31	$E_{\text{den}} = \frac{\epsilon E_f^2}{2}$	$E_f, \epsilon \in [1,5]$
II.10.9	$E_f = \frac{\sigma_{\text{den}}}{\epsilon}\frac{1}{1+\chi}$	$\sigma_{\text{den}}, \epsilon, \chi \in [1,5]$
II.11.28	$\theta = 1 + \frac{n\alpha}{1-n\alpha/3}$	$n, \alpha \in [0,1]$
II.13.23	$\rho_c = \frac{\rho_{c_0}}{\sqrt{1-v^2/c^2}}$	$\rho_{c_0} \in [1,5], v \in [1,2], c \in [3,10]$
II.15.4	$E = -\mu_M B\cos\theta$	$\mu_M, B, \theta \in [1,5]$
II.15.5	$E = -p_d E_f \cos\theta$	$p_d, E_f, \theta \in [1,5]$
II.24.17	$k = \sqrt{\frac{\omega^2}{c^2} - \frac{\pi^2}{d^2}}$	$\omega \in [4,6], c \in [1,2], d \in [2,4]$
II.27.16	$F_E = \epsilon c E_f^2$	$\epsilon, c, E_f \in [1,5]$
II.27.18	$E_{\text{den}} = \epsilon E_f^2$	$\epsilon, E_f \in [1,5]$
II.34.2a	$I = \frac{qv}{2\pi r}$	$q, v, r \in [1,5]$
II.34.2	$\mu_M = \frac{qvr}{2}$	$q, v, r \in [1,5]$
II.34.29a	$\mu_M = \frac{qh}{4\pi m}$	$q, h, m \in [1,5]$
II.37.1	$E = \mu_M(1+\chi)B$	$\mu, M, \chi \in [1,5]$
II.38.14	$\mu_S = \frac{\gamma}{2(1+\sigma)}$	$\gamma, \sigma \in [1,5]$
III.7.38	$\omega = \frac{2\mu_M B}{\hbar}$	$\mu_M, B, \hbar \in [1,5]$
III.8.54	$p_\gamma = \sin(\frac{Et}{\hbar})^2$	$E, t \in [1,2], h \in [1,4]$
III.12.43	$L = n\hbar$	$n, \hbar \in [1,5]$
III.15.12	$E = 2U(1-\cos(kd))$	$U, k, d \in [1,5]$
III.15.14	$m = \frac{\hbar^2}{2Ed^2}$	$\hbar, E, d \in [1,5]$
III.15.27	$k = \frac{2\pi\alpha}{nd}$	$\alpha, n, d \in [1,5]$
III.17.37	$f = \beta(1+\alpha\cos(\theta))$	$\beta, \alpha, \theta \in [1,5]$

4.3.2. Parameter settings

We set the maximum node number of GraphDSR to 6. The hidden size of the graph neural networks is 128 and the message passing step

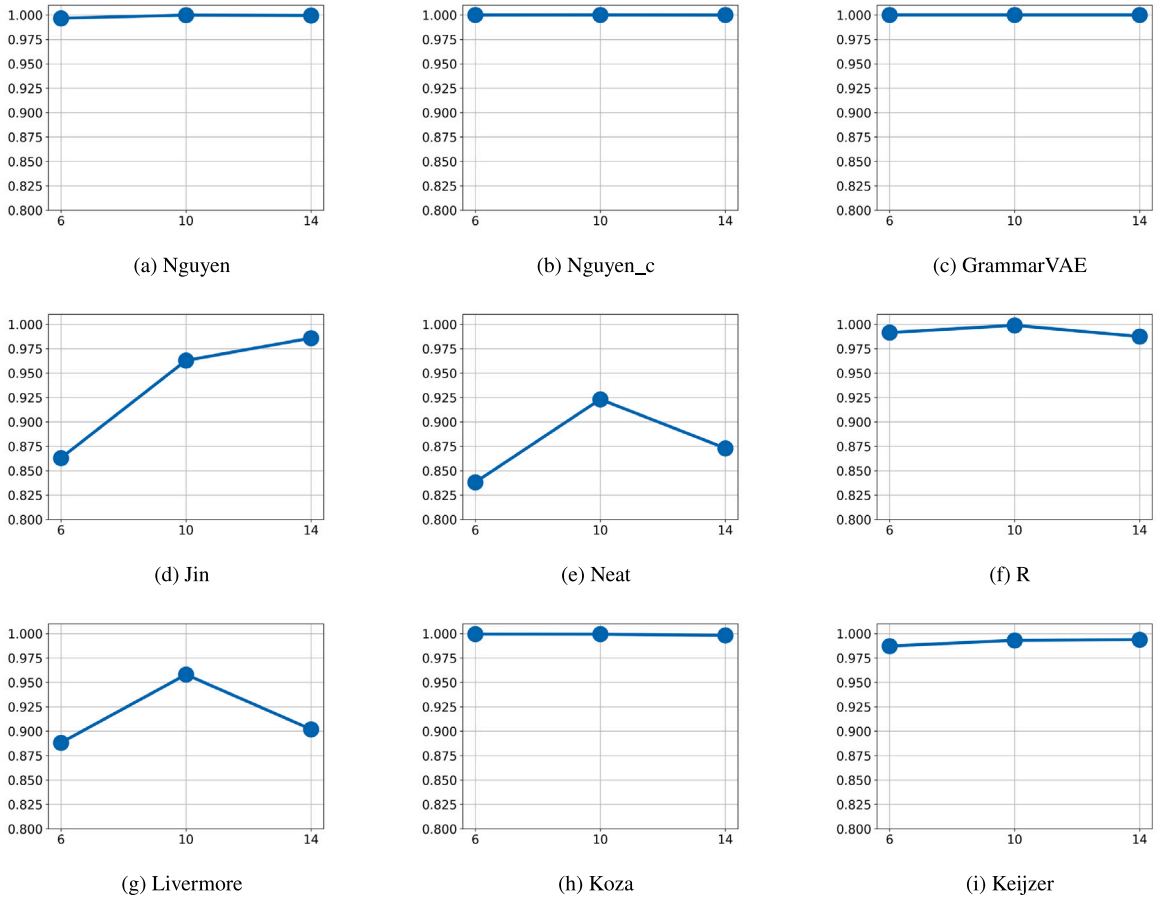

Fig. 3. Ablation study on the maximum node number. The x-axis is the maximum node number, the y-axis is the averaged R^2 within a dataset.

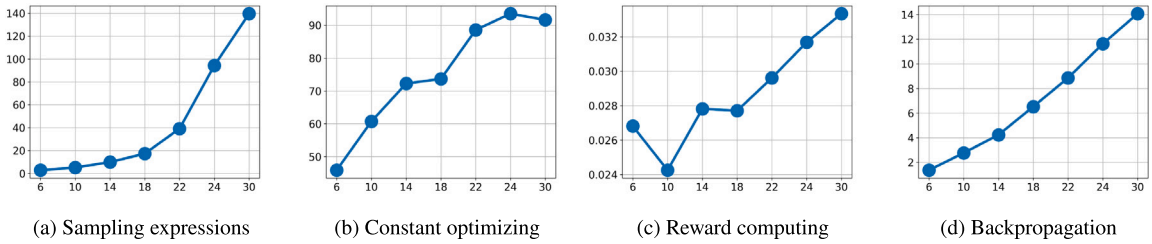

Fig. 4. Time cost analysis based on the maximum node number. The x-axis is the maximum node number, the y-axis is the averaged time cost for different phases. The exampled expression is "Nguyen1".

is 2. The learning rate for gradient descent is 0.001, the explore rate of reinforcement learning is 0.1, and the maximum iteration time is 100. For each epoch, we sample 200 expressions. The procedure is early-stopped if the reward exceeds 0.999.

To implement GP, we used the `GP-learn`[1] Python library; meanwhile, the other parameters were set to the default values. For BSR,[2] GP-GOMEA,[3] SBP-GP,[4] NeSymRes,[5] and SymFormer[6] we used the standard hyperparameters provided in the open-source implementations.

[1] https://gplearn.readthedocs.io/en/stable/index.html.
[2] https://github.com/ying531/MCMC-SymReg.
[3] https://github.com/marcovirgolin/GP-GOMEA/.
[4] https://github.com/marcovirgolin/GP-GOMEA.
[5] https://github.com/SymposiumOrganization/NeuralSymbolicRegressionThatScales.
[6] https://github.com/vastlik/symformer.

4.4. Results

4.4.1. Ablation study

The maximum R^2. We conducted an ablation study to evaluate the impact of varying the maximum node count in a DAG by setting this parameter to 6, 10, and 14. Restricting the maximum node number delineates the algorithm's search domain. We standardized the maximum iteration epoch at 100 and maintained consistency in the number of sampled expressions for each iteration. The findings are depicted in Fig. 3.

Analyzing benchmarks like Nguyen, Nguyen_c, GrammarVAE, Koza, and Keijzer, the R^2 performance metrics demonstrate commendable stability regardless of the node count variations. Conversely, the other benchmarks exhibited variable outcomes, and in some instances, performance deteriorated. We ascribe these inconsistencies to the expanded search domains attributed to increased node counts. Notably, as the node number escalates, the search space expands exponentially,

Table 3
The averaged maximum R^2 of each algorithms on public benchmarks.

Group	Datasets	GP	BSR	GP-GOMEA	SBP-GP	DSR	NeSymReS	SymFormer	GraphDSR
Base	Nguyen	0.9803	0.9916	0.9969	0.9993	0.9918	0.7659	0.9928	**0.9998**
	Nguyen_c	0.9876	0.9988	0.9972	0.9997	0.9984	0.8590	0.9996	**0.9999**
	GrammarVAE	**1.0000**	**1.0000**	**1.0000**	**1.0000**	**1.0000**	**1.0000**	**1.0000**	**1.0000**
	Jin	0.9835	0.7112	0.8985	0.9437	0.7638	0.4515	**0.9929**	0.9906
	Neat	0.5581	0.7923	0.7896	0.9080	0.7204	0.0000	**0.9422**	0.9230
	R	0.8060	0.9909	0.9513	0.9926	0.9417	0.0516	0.9815	**0.9999**
	Livermore	0.8248	0.7858	0.9512	0.9383	0.7649	0.6608	0.9435	**0.9581**
	Koza	0.5000	0.4999	0.6241	0.9627	0.9943	0.0000	0.9971	**0.9993**
	Keijzer	0.7240	0.8481	0.9870	0.9773	0.9512	0.7231	0.9946	**0.9952**
	Average	0.8671	0.8684	0.9513	0.9706	0.8955	0.6285	0.9799	**0.9852**
Feynman	Average	0.6148	0.6127	0.8630	0.8377	0.7230	0.5865	–	**0.9450**

necessitating a larger sample batch size and more iterations for optimal expression discovery. Therefore, in benchmarks such as Livermore and Neat, performance suffered, whereas for Jin, there was a noticeable improvement. We posit that this enhancement arises from the benchmark's complex data distributions, which demand intricate expressions for accurate fitting. For the benchmarks exhibiting stability, we credit this to their simpler data distributions where numerous equivalent expressions exist, at least within the trained interval. Overall, it is prudent to assess the complexity of data distributions and establish a correlative relationship with the maximum node count, a focus of our ongoing research. Moreover, as SR is not constrained by stringent time requirements — most applications being concentrated in scientific discovery (Wang et al., 2019) — it is feasible to experiment with various node counts and select the optimal one.

Time cost. We record the time cost of each phase during one single epoch, which contains sampling expressions, optimizing constants, computing rewards, and backpropagation for the policy net. The results are displayed in Fig. 4.

The exampled expression is "Nguyen1", considering the same sampling procedure and hyperparameter settings, the other expressions are in the same situation. We set the maximum node number as 6, 10, 14, 18, 22, 24, and 30, respectively, to see the effects of the maximum node number growth on the time cost. When the maximum node number is below a threshold, the main time cost comes from the constant optimizing that contains multiple inner training to determine the proper weight values. However, when it exceeds a threshold, such as 30, the sampling process takes more time than constant optimizing. While the time cost increases linearly for constant optimizing, reward computing, and backpropagation with the growth of maximum node number, the time cost of the expression sampling process grows exponentially. We attribute this to the one-by-one node and its edges sampling mechanism and the inner message-passing steps conducted by the GNN in the sampling process. However, we see that most of the tasks are well solved in a low maximum node number setting, which presents an affordable time cost. Plus, as argued before, SR is not a time-sensitive task, and based on the superior performances of GraphDSR, we think our method is better than these state-of-the-art methods.

4.4.2. Results on public benchmarks

Table 3 lists the experiment results of different methods on public datasets. Table 5 records the Inter Quartile Range across 10 independent runs. To save space, all the results are averaged within each dataset.

On the data group "Base", we achieve the highest averaged R^2. While the averaged R^2 across all benchmarks is very close to the results of SymFormer, as an end-to-end method, GraphDSR enjoys better adaptability. SymFormer cannot handle the task that contains a larger variable number of its training set. The evidence is found in the AI Feynman datasets which contain 3-variable equations but SymFormer can only process up to 2 variables based on its training set.

Table 4
Comparison with SymFormer on AI Feynman datasets.

Method	SymFormer		GraphDSR	
	R^2	IQR	R^2	IQR
Average	0.9926	–	**0.9981**	3.35×10^{-3}

Although the problem can be handled by generating a larger training set that contains enough variables to train the model, it still faces the problem that the model will not perform well on the out-of-training set. The main advantage of the large pre-trained model is its fast inference speed: once finished training, the search of the expression is super fast (could be in a few seconds). However, as argued above, SR is not a time-sensitive task. In contrast, GraphDSR keeps good adaptability across different number of variables.

On the AI Feynman dataset, we achieve the highest averaged R^2 with significant improvement. Note we display the comparison of SymFormer on Table 4 because SymFormer cannot return the search results of equations that contain 3 variables. We achieve very close performances against SymFormer on AI Feynman. Our method excels SymFormer on the flexibility of the variable number.

IQR, short for Interquartile Range, is an important statistical measure used to quantify the dispersion or spread of a dataset. It represents the range between the third quartile (Q3) and the first quartile (Q1) of a dataset, effectively capturing the spread of the middle 50% of the data. IQR is commonly used to identify and exclude outliers, as it focuses solely on the central portion of data, thereby minimizing the impact of extreme values on the overall analysis. From the Table 5, we can see that our method GraphDSR exhibits a low IQR at the 10^{-2} level. It indicates high consistency and compactness within the data, suggesting that most of the data points are clustered around the median with minimal variability. It also implies a low incidence of outliers and highlights the precision, indicating effective control and quality assurance in producing consistent outcomes. Note that, the IQR of the pre-trained models is in an extremely low level, because once the models are finished training, if the inputs are the same, the outputs would be the same. The pre-trained models show good robustness and consistency but exhibit poor adaptability when handling out-of-training tasks, which greatly limits the generality of the model.

4.4.3. Experiments on expressions that have reusable structures

We conduct experiments on expressions containing reusable structures to verify whether graph representation has an advantage in handling such problems. We use DSR as the counterpart because both DSR and GraphDSR use the same policy strategy updating the policy net and exploration strategy. Specifically, we generate 10 unary and 10 binary expressions, respectively. The R^2 are listed in Table 6.

GraphDSR shows higher R^2 compared with DSR which employs tree representation. Considering DSR uses tree representations and GraphDSR uses graph representations of mathematical expressions,

Table 5
The IQR of each algorithm.

Group	Datasets	GP	BSR	GP-GOMEA	SBP-GP	DSR	GraphDSR
Base	Nguyen	4.56×10^{27}	1.46×10^{-2}	6.79×10^{-3}	1.53×10^{-1}	1.92×10^{-2}	2.89×10^{-3}
	Nguyen_c	5.45×10^{27}	2.56×10^{-2}	4.66×10^{-3}	1.46×10^{-1}	1.47×10^{-2}	1.92×10^{-4}
	GrammarVAE	3.36×10^{-3}	4.61×10^{-4}	1.55×10^{-14}	1.02×10^{-1}	1.77×10^{-2}	6.89×10^{-7}
	Jin	**2.5688**	8.32×10^{5}	2.84×10^{-2}	2.08×10^{-1}	4.09×10^{-3}	1.89×10^{-1}
	Neat	2.99×10^{-1}	3.34×10^{-1}	2.80×10^{-2}	1.91×10^{-1}	5.71×10^{-3}	7.00×10^{-2}
	R	4.56×10^{-1}	1.87×10^{-1}	3.11×10^{-2}	1.79×10^{-1}	3.29×10^{-2}	1.92×10^{-2}
	Livermore	1.41×10^{34}	6.49×10^{1}	7.19×10^{2}	1.84×10^{-1}	1.61×10^{-2}	1.03×10^{-1}
	Koza	6.03×10^{29}	**5.0733**	**3.0232**	1.33×10^{-1}	3.28×10^{-2}	2.83×10^{-2}
	Keijzer	4.22×10^{25}	9.80×10^{1}	5.56×10^{-2}	1.94×10^{-1}	2.06×10^{-2}	2.37×10^{-2}
	Average	1.57×10^{33}	9.25×10^{4}	8.02×10^{1}	1.67×10^{-1}	1.82×10^{-2}	4.86×10^{-2}
Feynman	Average	2.25×10^{29}	4.78×10^{2}	3.37×10^{-1}	1.88×10^{-1}	5.82×10^{-2}	8.09×10^{-2}

Table 6
Compare with DSR in reusable expressions. The data is R^2.

Expressions	DSR		GraphDSR	
	R^2	IQR	R^2	IQR
Var1.1	0.9492	0.0158	**0.9759**	0.4721
Var1.2	0.9996	0.0261	**1.0000**	9.17×10^{-8}
Var1.3	0.0000	0.9582	**0.9998**	0.9988
Var1.4	0.8219	0.8182	**0.9997**	0.4582
Var1.5	**1.0000**	0.9281	**1.0000**	0.9968
Var1.6	0.9991	0.8584	**1.0000**	0.0084
Var1.7	0.9812	0.0185	**1.0000**	0.0005
Var1.8	0.7991	0.1584	**0.9454**	0.0923
Var1.9	0.5663	0.5565	**0.8054**	0.5531
Var1.10	0.9998	0.0313	**0.9999**	4.30×10^{-6}
Var2.1	0.0071	0.0070	**0.9864**	0.0306
Var2.2	0.8512	0.8437	**0.9914**	0.0065
Var2.3	0.1089	0.1064	**0.4096**	0.0752
Var2.4	**0.8152**	0.8039	0.0207	0.0772
Var2.5	0.9295	0.0413	**0.9365**	0.0101
Var2.6	0.2382	0.0002	**0.3895**	0.5733
Var2.7	0.7235	0.7192	**1.0000**	0.0847
Var2.8	0.4542	0.4357	**0.6562**	0.2702
Var2.9	0.9899	0.0031	**1.0000**	0.2606
Var2.10	**0.3482**	0.1821	**0.2231**	0.1719
Average	0.7270	0.3756	**0.8170**	0.2569

the results hint at an advantage of using graph-based representations. Graph-based representations capture complex relationships more effectively than tree structures in problems that have reusable structures.

5. Conclusion

In this paper, we introduce the method named GraphDSR which solves SR in graph-based mathematical expression representations. DAG is utilized as the representation of expression. The graph neural networks are applied to emit the node types' categorical and edge probability distribution, respectively. The GNNs could effectively grab the connecting relationship between different nodes by message passing. Then, the expression generation procedure is designed based on the node type of the adjacent matrix. For each expression, the first operator is sampled as the first (output) node, and the edges are determined. Then, determine the second node type and sample the edges. Repeat the step until reaches the maximum node number or the expression is completed. 4 domain-agnostic constraints are applied in each sampling step to reduce the search space. After sampling is completed, constant optimization is conducted to obtain the final expressions. The risk-seeking policy gradient is utilized to update the GNNs. We conduct experiments on 60 public benchmarks, 50 AI Feynman equations, and 20 self-generated reusable expressions. The results show the effectiveness of the proposed GraphDSR.

6. Future works

The learning strategy of GraphDSR is end-to-end, which belongs to the learning-from-scratch method. This type of method cannot utilize the experience of other search procedures or known data to expression map relationships. A natural consideration is to apply the pre-training of GNNs on large-scale training datasets. However, the graph is flexible and the map between the expression and its DAG is non-unique. This may bring challenges to the pre-training of the GNNs since multiple similar data distributions of the same expression could have multiple labels. Therefore, our future works aim to develop a set of rules to unique the mapping relationship between the expression and DAG. Then, apply these rules to pre-train the GNNs to get better performances.

CRediT authorship contribution statement

Jingyi Liu: Conceptualization, Investigation, Methodology, Software, Validation, Writing – original draft, Writing – review & editing. **Weijun Li:** Conceptualization, Funding acquisition, Methodology, Supervision. **Lina Yu:** Conceptualization, Funding acquisition. **Min Wu:** Conceptualization, Methodology, Software. **Wenqiang Li:** Methodology, Software. **Yanjie Li:** Methodology, Software. **Meilan Hao:** Methodology, Software.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. Single equation maximum R^2 of different methods on base benchmarks

See Table 7.

Appendix B. Single equation maximum R^2 of different methods on AI Feynman benchmarks

See Table 8.

Data availability

Data will be made available on request.

Table 7
The maximum R^2 of different methods on base benchmarks across 10 independent runs.

Expressions	GP	BSR	GP-GOMEA	SBP-GP	DSR	NeSymRes	SymFormer	GraphDSR
Nguyen1	0.9927	0.9999	**1.0000**	**1.0000**	**1.0000**	0.9159	**1.0000**	**1.0000**
Nguyen2	0.9982	0.9996	**1.0000**	**1.0000**	0.9998	0.8599	0.9999	**1.0000**
Nguyen3	0.9996	0.9996	0.9993	**1.0000**	0.9999	0.9548	0.9997	**1.0000**
Nguyen4	0.9954	0.9995	0.9995	0.9999	0.9979	0.9917	0.9944	**1.0000**
Nguyen5	0.9273	**1.0000**	0.9935	0.9990	0.9962	0.9880	**1.0000**	**1.0000**
Nguyen6	0.9984	**0.9995**	**0.9995**	**1.0000**	**0.9993**	0.9551	0.9987	**1.0000**
Nguyen7	0.9207	0.9864	0.9922	0.9972	0.9905	0.0000	0.9979	**1.0000**
Nguyen8	**1.0000**	0.9981	**1.0000**	**1.0000**	**1.0000**	0.9658	**1.0000**	**1.0000**
Nguyen9	**1.0000**	0.9983	**1.0000**	**1.0000**	0.9984	0.5349	0.9615	**1.0000**
Nguyen10	0.9967	**0.9994**	**1.0000**	**1.0000**	**1.0000**	0.8482	**1.0000**	**1.0000**
Nguyen11	**1.0000**	0.9240	**1.0000**	**1.0000**	**1.0000**	0.5059	**1.0000**	**1.0000**
Nguyen12	0.9355	0.9950	0.9797	0.9957	0.9203	0.6716	0.9616	0.9978
Nguyen2_	0.9973	0.9999	0.9974	0.9998	0.9984	0.6860	**1.0000**	**1.0000**
Nguyen5_	0.9896	**1.0000**	0.9861	0.9998	0.9933	0.9955	0.9985	**1.0000**
Nguyen8_	0.9998	0.9932	**1.0000**	**1.0000**	**1.0000**	0.7350	**1.0000**	**1.0000**
Nguyen8_2	**1.0000**	0.9996	**1.0000**	**1.0000**	**1.0000**	0.9350	**1.0000**	**1.0000**
Nguyen1_c	0.9978	0.9997	0.9997	0.9997	0.9987	0.9919	0.9997	**1.0000**
Nguyen5_c	0.9353	**1.0000**	0.9966	0.9995	0.9995	0.7076	**1.0000**	**1.0000**
Nguyen7_c	0.9704	0.9995	0.9967	0.9995	0.9974	0.9751	0.9989	**1.0000**
Nguyen8_c	**1.0000**	0.9992	**1.0000**	**1.0000**	**1.0000**	0.8622	**1.0000**	**1.0000**
Nguyen10_c	0.9984	0.9985	0.9987	0.9997	0.9983	0.8435	0.9995	**0.9999**
GrammarVAE1	**1.0000**	**1.0000**	**1.0000**	**1.0000**	**1.0000**	**1.0000**	**1.0000**	**1.0000**
Jin1	0.9960	0.9384	0.9963	0.9977	0.9927	0.1837	**1.0000**	0.9995
Jin2	0.9954	**1.0000**	0.9877	0.9907	0.9336	0.0000	**1.0000**	0.9692
Jin3	0.9635	**0.9995**	0.9644	0.9896	0.8444	0.6391	0.9976	0.9828
Jin4	0.9924	**1.0000**	**1.0000**	0.9983	0.8931	0.7296	**1.0000**	0.9924
Jin5	**1.0000**	0.0000	**1.0000**	**1.0000**	0.5749	**1.0000**	**1.0000**	**1.0000**
Jin6	0.9543	0.3297	0.4430	0.6865	0.3444	0.1571	0.9604	**1.0000**
Neat8	0.3534	0.6889	0.8096	0.9146	0.6324	0.0000	0.8873	**0.9502**
Neat9	0.7629	0.8958	0.7697	0.9016	0.8086	0.0000	**0.9971**	0.8960
R1	0.9904	0.9994	0.9899	0.9981	0.9927	0.0000	0.9998	**1.0000**
R2	0.7316	0.9896	0.8684	0.9818	0.8785	0.1551	0.9548	**0.9999**
R3	0.6961	0.9839	0.9958	0.9979	0.9539	0.0000	0.9899	**1.0000**
Livermore4	0.9998	**1.0000**	0.9999	**1.0000**	0.9996	0.0000	**1.0000**	**1.0000**
Livermore5	0.9983	0.9999	0.9993	**1.0000**	**1.0000**	0.8722	**1.0000**	**1.0000**
Livermore9	0.9932	0.9988	0.9937	0.9957	0.9837	0.8606	0.9943	**1.0000**
Livermore11	0.0000	0.0000	**1.0000**	0.2332	0.0710	0.0000	0.2182	0.4475
Livermore12	0.0000	0.0448	0.9992	**1.0000**	0.0000	**1.0000**	**1.0000**	0.9664
Livermore14	**1.0000**	0.9997	0.9997	**1.0000**	**0.9995**	0.0000	**0.9997**	0.9997
Livermore15	0.9989	0.9912	**1.0000**	**1.0000**	**1.0000**	0.9705	**1.0000**	**1.0000**
Livermore16	0.9999	0.9993	**1.0000**	**1.0000**	**1.0000**	0.7145	**1.0000**	**1.0000**
Livermore17	**1.0000**	0.0000	**1.0000**	**1.0000**	0.0527	**1.0000**	**1.0000**	**1.0000**
Livermore18	0.7444	0.9748	0.3278	0.9089	0.6322	0.8854	**1.0000**	**1.0000**
Livermore19	0.9961	0.9996	0.9995	**1.0000**	0.9986	0.7644	0.9994	**1.0000**
Livermore20	0.9879	**0.9999**	**1.0000**	**1.0000**	**1.0000**	**1.0000**	**1.0000**	**1.0000**
Livermore21	0.8300	0.9988	0.9988	0.9996	0.9727	0.1928	0.9979	**1.0000**
Livermore22	**1.0000**	0.9949	**1.0000**	**1.0000**	**1.0000**	0.9914	**1.0000**	**1.0000**
Koza2	**1.0000**	0.9999	0.9993	0.9993	0.9888	0.0000	**1.0000**	**1.0000**
Koza3	0.0000	0.0000	0.2491	0.9262	**0.9998**	0.0000	0.9942	0.9988
Keijzer3	0.0000	0.8374	**1.0000**	0.9145	0.9040	0.0000	0.9983	**1.0000**
Keijzer4	0.0000	0.0000	0.9759	0.9708	0.8108	0.4238	0.9954	0.9941
Keijzer6	0.9985	**1.0000**	**1.0000**	**1.0000**	**1.0000**	**1.0000**	**1.0000**	**1.0000**
Keijzer7	**1.0000**	0.9959	**1.0000**	**1.0000**	**1.0000**	**1.0000**	**1.0000**	**1.0000**
Keijzer9	**1.0000**	0.9998	0.9998	**1.0000**	0.9989	0.8020	**1.0000**	**1.0000**
Keijzer11	0.9475	0.9541	0.9566	0.9640	0.9494	0.9434	**0.9829**	0.9679
Keijzer14	0.9445	0.9995	0.9982	0.9992	0.9980	0.8090	0.9980	**0.9999**
Keijzer15	0.9019	0.9984	0.9657	0.9700	0.9490	0.8071	0.9824	**0.99986**

Table 8
The maximum R^2 of different methods on AI Feynman benchmarks across 10 independent runs.

Expressions	GP	BSR	GP-GOMEA	SBP-GP	DSR	NeSymRes	SymFormer	GraphDSR
I_6_2a	0.9778	0.9997	**1.0000**	0.9991	**1.0000**	0.8221	0.9997	**1.0000**
I_6_20	0.6671	0.7257	**0.9904**	0.9796	0.9158	0.0000	0.8978	0.9760
I_6_2b	0.5550	0.5154	**0.9188**	0.8545	0.6877	0.0000	–	0.7608
I_10_7	0.9996	0.9911	**0.9999**	0.9988	0.9977	0.9919	–	0.9997
I_12_1	**1.0000**	**1.0000**	**1.0000**	**1.0000**	**1.0000**	**1.0000**	**1.0000**	**1.0000**
I_12_4	0.1806	0.0000	**1.0000**	0.9640	0.3799	0.7637	–	0.9975
I_12_5	**1.0000**	**1.0000**	**1.0000**	**1.0000**	**1.0000**	**1.0000**	**1.0000**	**1.0000**
I_14_3	**1.0000**	**1.0000**	**1.0000**	**1.0000**	**1.0000**	**1.0000**	–	**1.0000**
I_15_10	0.9661	0.9960	0.9994	**0.9999**	0.9915	0.0000	–	0.9935
I_16_6	0.9469	**0.7697**	**0.9993**	**0.9481**	**0.8300**	0.7209	–	**0.9934**
I_18_12	**1.0000**	0.8195	**1.0000**	**1.0000**	**0.8855**	0.0655	–	**0.9671**
I_25_13	**1.0000**	0.6982	**1.0000**	**1.0000**	**1.0000**	**1.0000**	**1.0000**	**1.0000**
I_26_2	0.9812	0.9936	0.0000	**0.9990**	0.9913	0.9949	0.9929	0.9969
I_27_6	0.7359	**0.0000**	0.9866	0.7268	0.9392	**1.0000**	–	**1.0000**
I_29_4	**1.0000**	0.9133	**1.0000**	**1.0000**	**1.0000**	**1.0000**	**1.0000**	**1.0000**
I_30_3	0.0000	0.2866	**0.9536**	0.0000	0.5165	0.0000	–	0.4572
I_30_5	0.9975	0.9056	**1.0000**	**0.9998**	0.9956	0.9975	–	0.9993
I_34_10	0.0000	0.2005	**1.0000**	**1.0000**	0.0000	0.0000	–	0.5266
I_34_14	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	–	0.0000
I_34_27	0.9986	0.7896	**1.0000**	0.9997	**1.0000**	**1.0000**	**1.0000**	**1.0000**
I_37_4	0.8665	0.1992	0.9947	0.9282	0.7447	0.0917	–	**0.8439**
I_39_10	0.9959	**1.0000**	**1.0000**	0.0000	**1.0000**	**1.0000**	**1.0000**	**1.0000**
I_39_11	0.9214	0.5007	**0.9999**	0.9588	0.9526	0.8298	–	**1.0000**
I_43_31	**1.0000**	**0.7897**	**1.0000**	**1.0000**	**1.0000**	**1.0000**	–	**1.0000**
I_47_23	0.5199	**0.6862**	**1.0000**	**1.0000**	0.9613	0.6097	–	**0.9995**
I_48_20	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	–	0.0000
II_3_24	0.0000	0.0000	**1.0000**	**1.0000**	**1.0000**	**1.0000**	**1.0000**	**0.9999**
II_4_23	0.0000	0.4558	**1.0000**	**1.0000**	0.9376	0.6891	–	**1.0000**
II_8_7	0.990489	0.3378	**1.0000**	0.9724	0.9928	**1.0000**	–	**1.0000**
II_8_31	0.8460	**1.0000**	**1.0000**	**0.9991**	**1.0000**	**1.0000**	**1.0000**	**1.0000**
II_10_9	0.9976	0.1361	**1.0000**	**1.0000**	0.9286	0.6331	–	0.9911
II_11_28	0.9821	0.9884	**1.0000**	**1.0000**	0.9994	0.9950	0.9997	**1.0000**
II_13_23	0.9905	0.9979	**1.0000**	0.9998	0.9971	0.9962	–	**1.0000**
II_13_34	0.4013	0.9641	**0.9996**	0.9854	0.9982	0.6599	–	**0.9984**
II_5_4	0.7801	0.7351	**1.0000**	**1.0000**	**1.0000**	0.0000	–	**0.9994**
II_15_5	0.9599	0.4626	**1.0000**	0.6671	0.3531	0.0000	–	**0.9904**
II_24_17	**1.0000**	0.9838	0.9997	0.9962	0.9870	0.0000	–	0.9950
II_27_16	**1.0000**	0.7484	**1.0000**	**1.0000**	**1.0000**	**1.0000**	–	**1.0000**
II_27_18	0.2021	**1.0000**	**1.0000**	**1.0000**	**1.0000**	**1.0000**	**1.0000**	**1.0000**
II_34_2a	0.9923	0.5534	**1.0000**	0.9990	0.9286	**1.0000**	–	**1.0000**
II_34_2	0.4950	0.9996	**1.0000**	0.9999	0.9996	0.0000	–	**1.0000**
II_34_29a	**1.0000**	0.6399	**1.0000**	**1.0000**	**1.0000**	**0.0000**	–	**1.0000**
II_37_1	0.9461	0.8214	**1.0000**	**1.0000**	0.9809	0.8551	–	**1.0000**
II_38_14	0.9004	0.5885	**1.0000**	0.9981	0.9955	**1.0000**	0.9998	**1.0000**
III_7_38	0.0000	0.6972	0.0000	0.9965	0.9588	0.8210	–	**1.0000**
III_8_54	0.9579	0.0000	0.0000	0.0000	0.0000	0.0000	–	**0.9745**
III_12_43	0.0188	0.9885	**1.0000**	0.9967	**1.0000**	0.7204	**1.0000**	**1.0000**
III_15_12	0.0000	0.000000	**0.9754**	0.4012	0.3462	0.0000	–	0.9036
III_15_14	0.5568	0.3025	**1.0000**	0.9065	0.5044	**1.0000**	–	0.9978
III_15_27	0.4189	0.8574	**1.0000**	0.9958	0.0000	0.8990	–	**0.9535**
III_17_37	0.5158	0.6763	**1.0000**	0.6947	0.1277	0.0000	–	0.9668

References

Arnaldo, I., Krawiec, K., & O'Reilly, U.-M. (2014). Multiple regression genetic programming. In *Proceedings of the 2014 annual conference on genetic and evolutionary computation* GECCO '14, (pp. 879–886). New York, NY, USA: Association for Computing Machinery, http://dx.doi.org/10.1145/2576768.2598291.

Beadle, L., & Johnson, C. G. (2009). Semantically driven mutation in genetic programming. In *2009 IEEE congress on evolutionary computation* (pp. 1336–1342). http://dx.doi.org/10.1109/CEC.2009.4983099.

Cranmer, M. D., Xu, R., Battaglia, P. W., & Ho, S. (2019). Learning symbolic physics with graph networks. CoRR abs/1909.05862 URL http://arxiv.org/abs/1909.05862.

Fletcher, R. (2013). *Practical methods of optimization*. John Wiley & Sons.

Gilmer, J., Schoenholz, S. S., Riley, P. F., Vinyals, O., & Dahl, G. E. (2017). Neural message passing for quantum chemistry. In D. Precup, & Y. W. Teh (Eds.), *Proceedings of machine learning research*: *vol. 70*, *Proceedings of the 34th international conference on machine learning* (pp. 1263–1272). PMLR, URL https://proceedings.mlr.press/v70/gilmer17a.html.

Jin, Y., Fu, W., Kang, J., Guo, J., & Guo, J. (2019). Bayesian symbolic regression. arXiv preprint arXiv:2006.06666.

Kamienny, P.-A., Lample, G., Lamprier, S., & Virgolin, M. (2023). Deep generative symbolic regression with Monte-Carlo-tree-search. arXiv preprint arXiv:2302.11223.

Keijzer, M. (2003). Improving symbolic regression with interval arithmetic and linear scaling. In *Genetic programming: 6th European conference, euroGP 2003 essex, UK, April 14–16, 2003 proceedings* (pp. 70–82). Springer.

Kim, S., Lu, P. Y., Mukherjee, S., Gilbert, M., Jing, L., Čeperić, V., et al. (2021). Integration of neural network-based symbolic regression in deep learning for scientific discovery. *IEEE Transactions on Neural Networks and Learning Systems*, *32*(9), 4166–4177. http://dx.doi.org/10.1109/TNNLS.2020.3017010.

Koza, J. R. (1994). Genetic programming as a means for programming computers by natural selection. *Statistics and Computing*, *4*, 87–112.

Krawiec, K., & Pawlak, T. (2013). Approximating geometric crossover by semantic backpropagation. In *Proceedings of the 15th annual conference on genetic and evolutionary computation* (pp. 941–948).

La Cava, W., Spector, L., & Danai, K. (2016). Epsilon-lexicase selection for regression. In *Proceedings of the genetic and evolutionary computation conference 2016* (pp. 741–748).

Li, W., Li, W., Sun, L., Wu, M., Yu, L., Liu, J., et al. (2023). Transformer-based model for symbolic regression via joint supervised learning. In *The eleventh international conference on learning representations*. URL https://openreview.net/forum?id=ULzyv9M1j5.

Liao, R., Li, Y., Song, Y., Wang, S., Hamilton, W., Duvenaud, D. K., et al. (2019). Efficient graph generation with graph recurrent attention networks. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché Buc, E. Fox, R. Garnett (Eds.), *vol. 32*, *Advances in neural information processing systems*. Curran Associates, Inc., URL https://proceedings.neurips.cc/paper_files/paper/2019/file/d0921d442ee91b896ad95059d13df618-Paper.pdf.

Liu, J., Li, W., Yu, L., Wu, M., Sun, L., Li, W., et al. (2023). SNR: Symbolic network-based rectifiable learning framework for symbolic regression. *Neural Networks*, *165*, 1021–1034.

Lu, Q., Xu, C., Luo, J., & Wang, Z. (2022). AB-GEP: Adversarial bandit gene expression programming for symbolic regression. *Swarm and Evolutionary Computation*, *75*, Article 101197.

Luca, B., Tommaso, B., Alexander, N., Aurelien, L., & Giambattista, P. (2021). Neural symbolic regression that scales. In *Proceedings of machine learning research*: *vol. 139*, *Proceedings of the 38th international conference on machine learning* (pp. 936–945). PMLR.

McConaghy, T. (2011). FFX: Fast, scalable, deterministic symbolic regression technology. *Genetic Programming Theory and Practice IX*, 235–260.

Miller, J. F. (2020). Cartesian genetic programming: its status and future. *Genetic Programming and Evolvable Machines*, *21*, 129–168.

Mundhenk, T. N., Landajuela, M., Glatt, R., Santiago, C. P., Faissol, D. M., & Petersen, B. K. (2021). Symbolic regression via neural-guided genetic programming population seeding. CoRR abs/2111.00053 arXiv:2111.00053 URL https://arxiv.org/abs/2111.00053.

Pawlak, T. P., & Krawiec, K. (2018). Competent geometric semantic genetic programming for symbolic regression and boolean function synthesis. *Evolutionary Computation*, *26*(2), 177–212. http://dx.doi.org/10.1162/evco_a_00205.

Pawlak, T. P., Wieloch, B., & Krawiec, K. (2014). Semantic backpropagation for designing search operators in genetic programming. *IEEE Transactions on Evolutionary Computation*, *19*(3), 326–340.

Petersen, B. K., Larma, M. L., Mundhenk, T. N., Santiago, C. P., Kim, S. K., & Kim, J. T. (2021). Deep symbolic regression: Recovering mathematical expressions from data via risk-seeking policy gradients. In *International conference on learning representations*. URL https://openreview.net/forum?id=m5Qsh0kBQG.

Tenachi, W., Ibata, R., & Diakogiannis, F. I. (2023). Deep symbolic regression for physics guided by units constraints: toward the automated discovery of physical laws. arXiv preprint arXiv:2303.03192.

Udrescu, S.-M., Tan, A., Feng, J., Neto, O., Wu, T., & Tegmark, M. (2020). AI feynman 2.0: Pareto-optimal symbolic regression exploiting graph modularity. *Advances in Neural Information Processing Systems*, *33*, 4860–4871.

Udrescu, S.-M., & Tegmark, M. (2020). AI feynman: A physics-inspired method for symbolic regression. *Science Advances*, *6*(16), eaay2631.

Udrescu, S.-M., & Tegmark, M. (2021). Symbolic pregression: Discovering physical laws from distorted video. *Physical Review E*, *103*(4), Article 043307.

Uy, N. Q., Hoai, N. X., O'Neill, M., McKay, R. I., & Galván-López, E. (2011). Semantically-based crossover in genetic programming: application to real-valued symbolic regression. *Genetic Programming and Evolvable Machines*, *12*, 91–119.

Valipour, M., You, B., Panju, M., & Ghodsi, A. (2021). Symbolicgpt: A generative transformer model for symbolic regression. arXiv preprint arXiv:2106.14131.

Vastl, M., Kulhánek, J., Kubalík, J., Derner, E., & Babuska, R. (2022). SymFormer: End-to-end symbolic regression using transformer-based architecture. http://dx.doi.org/10.48550/arXiv.2205.15764, CoRR abs/2205.15764 arXiv:2205.15764.

Virgolin, M., Alderliesten, T., & Bosman, P. (2019a). Linear scaling with and within semantic backpropagation-based genetic programming for symbolic regression. In *The genetic and evolutionary computation conference*.

Virgolin, M., Alderliesten, T., & Bosman, P. A. (2019b). Linear scaling with and within semantic backpropagation-based genetic programming for symbolic regression. In *Proceedings of the genetic and evolutionary computation conference* (pp. 1084–1092).

Virgolin, M., Alderliesten, T., Witteveen, C., & Bosman, P. (2021). Improving model-based genetic programming for symbolic regression of small expressions. MIT Press One Rogers Street, Cambridge, MA 02142-1209 USA Journals-Info@Mit.Edu.

Walker, J. A., & Miller, J. F. (2004). Evolution and acquisition of modules in cartesian genetic programming. In *European conference on genetic programming* (pp. 187–197). Springer.

Walker, J. A., & Miller, J. F. (2008). The automatic acquisition, evolution and reuse of modules in cartesian genetic programming. *IEEE Transactions on Evolutionary Computation*, *12*(4), 397–417.

Wang, Y., Wagner, N., & Rondinelli, J. M. (2019). Symbolic regression in materials science. *MRS Communications*, *9*(3), 793–805.

Zhang, H., Zhou, A., Qian, H., & Zhang, H. (2022). PS-tree: A piecewise symbolic regression tree. *Swarm and Evolutionary Computation*, *71*, Article 101061.